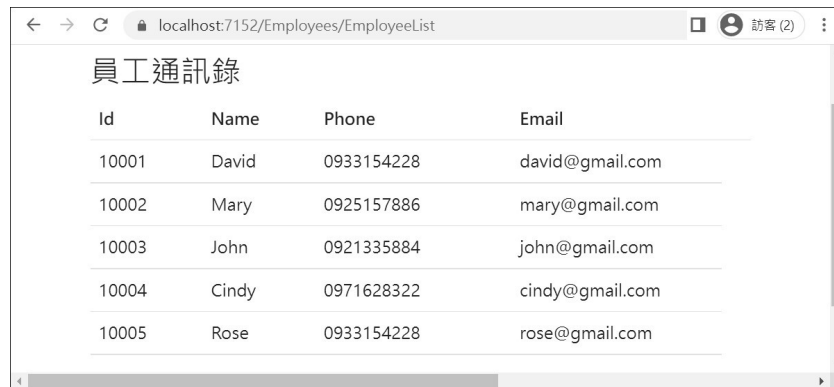


```
new Employee {Id=10005, Name="Rose", Phone="0933-154228",Email="rose@gmail.com"}  
};
```

以上 `List<Employee>` 是泛型集合，內有五個 `Employee` 型別物件（代表五筆資料），而 `employees` 就代表 `model` 物件，將 `employees` 以 `Model` 方式傳遞給 `View` 顯示。

範例 5-2 製作員工通訊錄列表

以下說明如何製作員工通訊錄。



Id	Name	Phone	Email
10001	David	0933154228	david@gmail.com
10002	Mary	0925157886	mary@gmail.com
10003	John	0921335884	john@gmail.com
10004	Cindy	0971628322	cindy@gmail.com
10005	Rose	0933154228	rose@gmail.com

圖 5-8 員工通訊錄列表

step 01 建立 Employee 的 Model 模型

在 `Action` 建立 `List<Employee>` 泛型集合前，需先建立 `Employee` 模型，在 `Models` 資料夾按滑鼠右鍵→加入 `Employee.cs` 類別及四個屬性

Models/Employee.cs

```
public class Employee  
{  
    public int Id { get; set; }  
    public string Name { get; set; }  
    public string Phone { get; set; }  
    public string Email { get; set; }  
}
```

Model 模型定義

step02 在 Controllers 資料夾按滑鼠右鍵→【加入】→【控制器】→【MVC 控制器-空白】→【加入】→命名為「EmployeesController」→【新增】

step03 在 Controller 引用 Model 命名空間，Controller 程式才能參考到 Models 資料夾的 Employee 類別

```
using Mvc7_Pillars.Models;
```

或在 Globalusings.cs 中加入（建議此方式）

```
global using Mvc7_Pillars.Models;
```

step04 建立 model 資料物件

在 Employees 控制器的 EmployeeList()方法中建立 model 物件，以 List<Employee> 泛型集合建立員工資料

 Controllers/EmployeesController.cs

```
public class EmployeesController : Controller
{
    public IActionResult EmployeeList()
    {
        List<Employee> employees = new List<Employee>
        {
            new Employee {Id=10001, Name="David", Phone="0933154228",
                Email="david@gmail.com"},
            new Employee {Id=10002, Name="Mary", Phone="0925157886", Email="mary@gmail.com"},
            new Employee {Id=10003, Name="John", Phone="0921335884", Email="john@gmail.com"},
            new Employee {Id=10004, Name="Cindy", Phone="0971628322",
                Email="cindy@gmail.com"},
            new Employee {Id=10005, Name="Rose", Phone="0933154228", Email="rose@gmail.com"}
        };
        return View(employees);
    }
}
```

Diagram annotations:

- Action 方法 (points to EmployeeList())
- List<Employee> 的 model 物件 (points to employees)
- 將 model 物件傳入 View 中 (points to View(employees))

說明：以上「集合初始化設定」在 2-12 小節曾介紹過

step 05 利用 Scaffolding 範本及 Model 模型類別產出 View 檢視樣板

在這借助 Scaffolding 自動產生程式樣板，快速建立出 View 檢視。
 在 EmployeeList() 方法按滑鼠右鍵→【新增檢視】→【Razor 檢視】→範本選擇【List】→模型類別「Employee」→模型內容類別「FriendContext」→【新增】



圖 5-9 選擇 View 檢視的範本及模型類別

step 06 下面是自動產生的 View 樣板，檔名結尾是.cshtml，它就是 Razor View，裡面是 HTML 和 C#兩種語法混合體，而 C#指令前面一定要加@符號

 Views/Employees/EmployeeList.cshtml

```
@model IEnumerable<Mvc7_Pillars.Models.Employee>

@{
    ViewData["Title"] = "EmployeeList";
}

<h2>員工通訊錄</h2>
@*<p>
    <a asp-action="Create">Create New</a>
</p>*@
<table class="table">
```

用@model 指示詞明確宣告傳入 model 物件型別

註記掉

Anchor Tag Helper

```

<tr>
  <th>
    @Html.DisplayNameFor(model => model.Id)
  </th>
  <th>
    @Html.DisplayNameFor(model => model.Name)
  </th>
  <th>
    @Html.DisplayNameFor(model => model.Phone)
  </th>
  <th>
    @Html.DisplayNameFor(model => model.Email)
  </th>
</tr>
@foreach (var item in Model) {
  <tr>
    <td>
      @Html.DisplayFor(modelItem => item.Id)
    </td>
    <td>
      @Html.DisplayFor(modelItem => item.Name)
    </td>
    <td>
      @Html.DisplayFor(modelItem => item.Phone)
    </td>
    <td>
      @Html.DisplayFor(modelItem => item.Email)
    </td>
    @*<td>
      <a asp-action="Edit" asp-route-id="@item.Id">Edit</a> |
      <a asp-action="Details" asp-route-id="@item.Id">Details</a> |
      <a asp-action="Delete" asp-route-id="@item.Id">Delete</a>
    </td>*@
  </tr>
}
</table>

```

DisplayNameFor 方法是用來顯示 Model 標題名稱
 用 foreach 逐一取出 Model 中 Item 項目
 DisplayFor 方法是用來顯示 Item 項目值
 註記掉
 指定 Action
 指定路由 id 參數

說明：

1. @model 是用來宣告強型別檢視，好處是支援 IntelliSense 及編譯時檢查

2. `IEnumerable<Mvc7_Pillars.Models.Employee>` 會針對傳入的 `List<Employee>` 集合公開一個列舉器
3. `Html.DisplayNameFor(model => model.Id)` 指令，其中 `Html` 是指 `HTML Helpers`，`DisplayNameFor` 是 `HTML Helpers` 支援的眾多方法之一，用來顯示 `Model` 的標題，`model => model.Id` 是 `Lambda` 表示式
4. `Html.DisplayFor(modelItem => item.Id)` 是用來顯示 `Model` 項目值
5. 原本自動產生的 `View` 沒有顯示 `Id` 欄位，需要的話可自行補上
6. `Anchor` 超連結中的 `asp-action` 是指定 `Action` 動作方法，`asp-route-id` 是指定路由 `id` 參數，它是 `Anchor Tag Helper` 標籤協助程式的語法
7. `Tag Helpers & HTML Helpers` 指令詳細用法在第十及十一章會介紹

5-4-2 強型別檢視和動態型別檢視之區別

`Action` 以 `Model` 方式傳遞資料給 `View`，而在 `View` 這端，卻會因是否用 `@model` 指示詞具體指明傳入的 `Model` 型別為何，而有強型別檢視（`Strongly Typed View`）與動態型別檢視（`Dynamic Typed View`）之差異。

像之前寵物店例子，`Action` 傳遞 `Model` 給 `View`，在 `View` 開頭處並未使用 `@model` 指示詞宣告型別，那麼這個 `View` 就會是動態型別檢視，而缺點是效能較差、不支援 `IntelliSense` 和編譯時期型別檢查。較好的做法是在開頭用 `@model` 宣告傳入 `model` 物件型別，`View` 就會是強型別檢視：